

Software Quality Attributes Dataset - Yapay Zeka Modelleme Stratejisi

Ders: Yazılım Test Sürecinin Optimizasyonu İçin Yapay Zeka Yöntemleri

Bu belge, analiz edilen **SoftwareQualityAttributesDataset** veri seti üzerinden makine öğrenmesi ve yapay zeka modelleri geliştirmek için izlenebilecek stratejileri, potansiyel hedef değişkenleri, eğitim adımlarını ve sektörel uygulama örneklerini detaylandırmaktadır.

1. Hedef Değişken Belirleme (Target Variable)

Ana Hedef: **GodClass** (İkili Sınıflandırma - Binary Classification)

Veri setindeki en doğal ve en kıymetli hedef değişken **GodClass** sütunudur. Hedefimiz: *Bir Java sınıfının verilen metriklerine (CBO, RFC, LCOM, WMC vb.) bakarak o sınıfın bir "God Class" (aşırı büyümüş, çok fazla iş yapan, bakımı zor, anti-pattern sınıf) olup olmadığını tahmin etmektir.*

- Sınıflar: **0 (God Class Değil)**, **1 (God Class)**

Alternatif Hedef: **mdi_godclass** (Regresyon / Olasılık Tahmini)

Eğer ikili sınıflandırma yerine, *bu sınıfın bir God Class olmaya ne kadar yatkın olduğunu (risk skorunu)* ölçmek istiyorsanız, **mdi_godclass** sürekli değişkenini hedefleyen bir regresyon modeli (ör. Random Forest Regressor) eğitebilirsiniz.

2. Kullanılabilecek Model Türleri

Analiz raporunda (Bölüm 3 ve 9), metriklerin uç değerler (outlier) barındırdığı ve sağa çarpık (skewed) dağılımlara sahip olduğu görüldü. Bu yüzden standart mesafe tabanlı algoritmalarla ziyade ağaç tabanlı algoritmalar çok daha iyi performans gösterecektir.

1. Ağaç Tabanlı Topluluk Modelleri (Tree-based Ensemble) - ★ En Yüksek Öneri

- XGBoost & LightGBM:** Verideki dengesizlikle (imbalance) çok daha iyi başa çıkarlar. Outlier'lara karşı dirençlidirler ve yüksek boyutta çok hızlı öğrenirler.
- Random Forest:** Aşırı öğrenmeye (overfitting) daha dayanıklıdır ve hangi metriklerin daha önemli olduğunu (Feature Importance) kolayca söyler.

2. Derin Öğrenme (Deep Learning) Modelleri

- Multi-Layer Perceptron (MLP):** Yüksek ölçekli (190,000+ satırlık) bu veri setinde, iyi optimize edilmiş ve verileri **RobustScaler** ile ölçeklendirilmiş bir yapay sinir ağı test optimizasyonu için güçlü bir model olabilir.

3. Temel Yöntemler (Baseline Modeller)

- Lojistik Regresyon (Logistic Regression) / SVM:** Referans performans değeri elde etmek için kullanılmalıdır. Outlier'lar için **RobustScaler** uygulanması zorunludur.

3. Eğitimi Adımları ve Stratejiler

Model eğitiminde izlenmesi gereken best-practice (en iyi pratik) adımlar şunlardır:

Adım 1: Veri Ön İşleme ve Temizlik (Preprocessing)

- **Gereksiz Sütunların Atılması:** Tamamen boş olan veya sızma (data leakage) yaratacak sütunların çıkarılması. **Coverage**, **#C** vb. paket metrikleri ile **WMC.1**, **LOC.1** gibi tekrarlayan (korelasyonu 1.0) sütunların droplanması.
- **Kategorik Kodlama (Encoding):** **Complexity**, **Size** vb. low/medium/high değerleri sıralı olarak (0,1,2 veya One-Hot-Encoding ile) sayısal formata dönüştürülmelidir.
- **Eksik Verilerin Giderilmesi:** %0.03 gibi çok küçük olan NA değerler silebilir veya medyana (median) göre doldurulabilir.

Adım 2: Çapraz Proje Bölünmesi (Cross-Project Defect Prediction Scenario)

Klasik **%80 Eğitim - %20 Test** ayrımı yerine yazılım mühendisliğinde çok daha kıymetli bir test stratejisi vardır: **Cross-Project Validation**. 10 adet proje var. **Strateji:** Modeli 9 projeyle eğitip, daha önce hiç görmediği 1 proje (örneğin Hadoop) üzerinde test ederek gerçek dünya performansını ölçmek.

Adım 3: Sınıf Dengesizliğini (Imbalance) Çözme

Veri setinde GodClass oranı %17.11'dir (~1:5 oran). Model sürekli "0" (God Class Değil) demeye yatkın olacaktır.

- **SMOTE (Synthetic Minority Over-sampling Technique):** Azınlık sınıfı için sentetik veri üreterek dengelemek. Hedef değışkonde %50-%50 oluşturmak.
- **Algoritmik Ağırlıklandırma:** XGBoost veya Random Forest kullanırken **scale_pos_weight** veya **class_weight='balanced'** argümanlarını açmak.

Adım 4: Model Değerlendirmesi İçin Doğru Metrikler

Veri dengesiz olduğu için metrik olarak **Accuracy (Doğruluk)** kesinlikle yanıltıcı olur.

- Kullanılması gereken temel metrikler: **F1-Score**, **Precision (Hassasiyet)**, **Recall (Duyarlılık)**
- **PR-AUC (Precision-Recall Area Under Curve):** God Class gibi anti-patternleri yakalamak istiyorsak Receiver Operating Characteristic (ROC)'tan ziyade PR-AUC skoruna güvenmeliyiz.

4. Uygulama Örnekleri ve Optimizasyon Senaryoları (Use Cases)

Eğitilen bu God Class tespiti modelini gerçek bir yazılım organizasyonuna şu şekilde oturtabiliriz:

Senaryo 1: CI/CD Pipeline (Code Review) Entegrasyonu 🚀

Süreç: Yazılımcılar yeni özellik (feature) geliştirip Git'e "Push" veya "Pull Request (PR)" açtığında süreç tetiklenir. **Yapay Zeka Etkisi:** Pipeline arkasında çalışan statik analiz motorları, sınıfların kod metriklerini anlık olarak ölçer (LOC, WMC, CBO). Çıkan veriler eğittiğimiz modele sokulur. Eğer model bir sınıfı **1 (God Class Olarak Evriliyor)** olarak işaretlerse, PR otomatik bloğa alınır ve yazılımcılara: *"Tahminlerimize göre UserService sınıfını GodClass anti-patternine sürüklediniz. Lütfen refactoring yaparak sınıfları bölün"* uyarısı verir.

Senaryo 2: Technical Debt (Teknik Borç) Planlaması ve Önceliklendirme 🧩

Süreç: Milyonlarca satıra ulaşmış Legacy (eski) kod mimarilerinde iyileştirme için nereden başlanacağı bilinemez. **Yapay Zeka Etkisi:** Eğittiğimiz model, projeyi tarayarak hangi sınıfların God Class olduğunu tespit eder. Sonra özellik önem sırasına (Feature Importance) bakar. Hangi sınıfların (örneğin WMC - Karmaşıklığı aşırı yüksek olanlar) hemen müdahale edilmesi gerektiğini ortaya çıkararak sprint planlamasına veri sağlar.

Senaryo 3: Risk Bazlı Test (Risk-Based Testing) Otomasyonu

Süreç: Tüm projeyi her commit sonrasında test etmek uzun sürer. **Yapay Zeka Etkisi:** Test süreçlerini optimize etmek için hedef değişken yerine regresyon modeli (`mdi_godclass`) kullanılarak her sınıf için 0 ile 1 arasına bir risk skoru atanır. Yüksek God Class potansiyeli gösteren (ve sürekli değişen) modüller test otomasyonlarında **"Öncelikli Koşulacak Testler Seti"** içine yerleştirilir. Böylece test eforu %100'den, en riskli olan %20'lik koda kaydırılarak ciddi bir test eforu / süresi optimizasyonu yapılır.